BOARD VERIFICATION SIGN-OFF
Student has programmed the DE1, verified the 5-bit ALU on hardware, recorded results in the table above, and completed the reflection questions.

Instructor signature: Asst. Dr. Tamnuwat Date: 2026-10-27 Marks awarded: 69 / 69

Lab 4 — Combinational Logic: Multiplexers, Decoders, Priority Encoders & a 5-bit ALU

Name: _____	**Student ID:** _____

Learning Objectives	By the end of this lab, you will be able to: **(1)** build multiplexer and decoder logic using assign and case constructs, **(2)** apply if-else and case statements correctly, avoiding unintended latches, **(3)** predict and explain how a case-based select structure behaves, in writing, **(4)** design a priority encoder from a written specification, and **(5)** implement and verify a 5-bit ALU on the DE1 board using switches and push buttons.

Part A · Multiplexers & Decoders (Exercises 1–2)

FILL-IN-BLANK	**Exercise 1 — 4-to-1 Multiplexer (Complete the Blanks)**	**5 pts/_____**

How a 4-to-1 Mux behaves:
Inputs: in0, in1, in2, in3 (1-bit each), plus a 2-bit select sel[1:0]
Output: y (1-bit) — copies whichever input sel points to
Think of sel as an address: it picks exactly one of the four inputs to pass through to y.

sel[1:0]	y =
00	in0
01	in1
10	in2
11	in3

Complete the code below. A **4-to-1 multiplexer** selects one of four 1-bit inputs based on a 2-bit select signal. Fill in every ___ — there are **5 blanks** to fill.

```
module mux4to1 (
  input  wire in0, in1, in2, in3,
  input  wire [1:0] _________________, // 1. select signal name
  output ____reg_____________ y      // 2. keyword for an always-driven output
);
  always @(*) begin
    case (_____sel_____________)      // 3. switch on? (same name as blank 1)
      2'b00: y = in0;
      2'b01: y = in1;
      2'b10: ___y = in2_____________________; // 4. complete this branch
      2'b11: y = in3;
      default: y = ___1'b0_____________;   // 5. a safe default value
    endcase
  end
endmodule
```

DEBUG	**Exercise 2 — Find 5 Bugs in This 2-to-4 Decoder Code**	**10 pts/_____**

How a 2-to-4 Decoder behaves:

Input: sel[1:0]

Output: out[3:0] — one-hot: exactly one bit is 1, matching the binary value of sel

Example: sel = 2'b10 → out = 4'b0100 (bit 2 set, all others 0).

sel[1:0]	out[3:0]
00	0001
01	0010
10	0100
11	1000

The module below *should* implement a 2-to-4 decoder but it has **exactly 5 bugs**. Circle or underline each bug in the listing, then fill in the table.

```
module decoder2to4 (
  input  [1:0] sel,
  output [3:0] out                  // is this declaration complete?
);
  always @(sel[0]) begin            // is the sensitivity list complete?
    case (sel)
      2'd0: out = 4'b0001;
      2'd0: out = 4'b0010;          // check this case label carefully
      2'd2: out = 4'b0010;          // check this constant carefully
    endcase
  end
endmodule
```

Bug log:

Bug #	Line / Location	What is wrong?	Corrected code
1	Line 3: output [3:0] out	Missing 'reg' type for always-driven output	output reg [3:0] out
2	Line 5: always @(sel[0])	Incomplete sensitivity list (missing sel[1])	always @(*)
3	Line 8: 2'd0: out = 4'b0010;	Duplicate case label (2'd0 appears twice)	2'd1: out = 4'b0010;
4	Line 9: 2'd2: out = 4'b0010;	Wrong one-hot constant for sel=2 (bit 1 set)	2'd2: out = 4'b0100;
5	Lines 9-10: case end	Missing 2'd3 branch & default (inferred latch)	2'd3: out=4'b1000; default: out=4'b0000;

Part B · Control Constructs & Reasoning (Exercises 3–5)

MATCH	Exercise 3 — Match the Construct to Its Behavior	5 pts/_____

Write the matching letter in the Answer column.

Construct	Answer	Behavior	Letter
if with no final else	E	Parallel mux — every branch has equal priority	A
case with no default	D	Priority mux — the first true condition wins	B
if-else chain	B	Don't-care matching, useful for priority decode	C
casez	C	Risks an inferred latch in a case statement	D
case (full)	A	Also risks an inferred latch, in an if statement	E

PREDICT	**Exercise 4 — Predict the Output (No Simulator Needed)**	**5 pts/_____**

Given the always block below, with a = 4'b0110 (6) and b = 4'b0011 (3) fixed, predict result for each op value.

```
always @(*) begin
  case (op)
    3'b000: result = a + b;
    3'b001: result = a - b;
    3'b010: result = a & b;
    3'b011: result = a | b;
    default: result = 4'b0000;
  endcase
end
```

op	Operation	Predicted result (binary)	Decimal
000	a + b	4'b1001	9
001	a – b	4'b0011	3
010	a & b	4'b0010	2
011	a | b	4'b0111	7
101	(not listed → default)	4'b0000	0

SHORT ANSWER	**Exercise 5 — Why Is an ALU Just a Big Multiplexer?**	**4 pts/_____**

In 3–5 sentences, answer the following:

(a) What signal selects which branch of the case statement runs?

(b) Why is a case statement a more natural fit here than a long if-else chain?

(c) What should happen if the opcode value doesn't match any defined branch, and why does that matter in real hardware?

(a) แต่ละบล็อกฟังก์ชัน (Adder, Subtractor, Logic) คำนวณอินพุต A และ B ขนานพร้อมกันตลอดเวลา

(b) สัญญาณ Opcode ทำหน้าที่เป็น Select line เลือกผลลัพธ์ของฟังก์ชันที่ต้องการเพียงค่าเดียวส่งออก

(c) โครงสร้างนี้ตรงกับฮาร์ดแวร์ Multiplexer ขนานกันจริงที่เชื่อมวงจรคำนวณเข้ากับพอร์ตเอาต์พุต

Part C · Module Specification (Exercise 6)

DESIGN	Exercise 6 — Write a Priority Encoder from Specification	8 pts/_____

Using the specification below, write the complete 4-to-2 priority encoder module. Use an `always` block with `if-else` only — no `case` statement, and no structural instantiation.

Port	Direction	Width	Behavior
in	input	[3:0]	4 request lines; in[3] is highest priority, in[0] lowest
val	output	1 bit	val = 1 if any input is asserted, else 0 (hint: OR-reduction |in)
code	output	[1:0]	Binary index of the highest-priority asserted input

Behavior & design rules:

1. Highest index wins — if in[3] is 1, code = 2'b11 regardless of the other bits.
2. Check from the top down — in[3], then in[2], then in[1], then in[0], using if-else if.
3. No requests asserted — val = 0 and code defaults to 2'b00.

Write your complete priority_encoder module in the space below:

```
// priority_encoder.v — write your code here

    module priority_encoder (
       input  wire [3:0] in,    // 4 request lines (in[3] highest)
       output reg        val,   // 1 if any input asserted
       output reg  [1:0] code   // Binary index of highest priority
    );

       always @(*) begin
          if (in[3]) begin
             val  = 1'b1;  code = 2'b11; // in[3] active -> 3
          end else if (in[2]) begin
             val  = 1'b1;  code = 2'b10; // in[2] active -> 2
          end else if (in[1]) begin
             val  = 1'b1;  code = 2'b01; // in[1] active -> 1
          end else if (in[0]) begin
             val  = 1'b1;  code = 2'b00; // in[0] active -> 0
          end else begin
             val  = 1'b0;  code = 2'b00; // no request
          end
       end

    endmodule
```

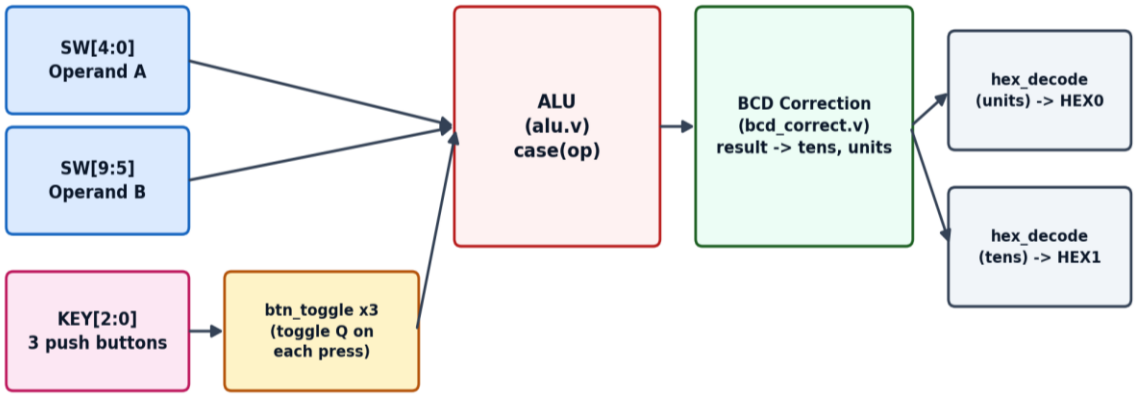
Part D · Integrated FPGA Project (Exercises 7–8) — Real Circuit, Real Board

Mission Brief — 5-bit ALU on the DE1

Exercises 7 and 8 form one integrated project. You will build an **ALU** that computes one of 8 operations on two 5-bit operands, selected by a 3-bit opcode, and displays the **decimal** result via BCD correction. **No system clock is used — the opcode buttons toggle their own state, and the ALU itself is pure combinational logic. Program the board and demonstrate to your instructor.**

Inputs: `SW[4:0]` = A, `SW[9:5]` = B (switches) · `KEY[2:0]` = 3 push buttons, each toggles one Op bit on press | **Output:** `HEX1:HEX0` = result in decimal (tens, units)

5-bit ALU Block Diagram (no system clock — buttons self-clock their own toggle)



Block Diagram. SW[4:0] and SW[9:5] feed the ALU directly. Each of the 3 push buttons in KEY[2:0] drives its own `btn_toggle` module, which flips a stored bit every time its button is pressed — those 3 stored bits form op[2:0]. The ALU's 6-bit result goes through BCD correction to get tens/units digits, and two hex_decode units drive HEX1:HEX0.

Given files — do NOT modify. Add de1_top.v, btn_toggle.v, and hex_decode.v to your Quartus project along with your own alu.v and bcd_correct.v.

```
// btn_toggle.v — given, for reference only. No system clock: the
module btn_toggle (
  input  key_n,   // active-low button
  output reg q,
  output     led
);
  always @(negedge key_n)   // pressing the button creates the edge
    q <= ~q;
  assign led = q;
endmodule
```

```
// hex_decode.v — given, for reference only (digits 0-9, active-format matches Lab 3)
module hex_decode (input [3:0] value, output reg [6:0] seg); // {g,f,e,d,c,b,a}
  always @(*) begin
    case (value)
      4'd0: seg=~7'b0111111;
      4'd1: seg=~7'b0000110;
      4'd2: seg=~7'b1011011;
      4'd3: seg=~7'b1001111;
      4'd4: seg=~7'b1100110;
      4'd5: seg=~7'b1101101;
      4'd6: seg=~7'b1111101;
      4'd7: seg=~7'b0000111;
      4'd8: seg=~7'b1111111;
      4'd9: seg=~7'b1101111;
      default: seg=~7'b0000000;
    endcase
  end
endmodule
```

```
module top (
  input  [9:0] SW,          // SW[4:0]=A, SW[9:5]=B
  input  [2:0] KEY,         // 3 push buttons -> toggle to form Op[2:0]
  output [2:0] LEDG,        // lights up to show each Op bit's current state
  output [6:0] HEX1, HEX0   // decimal result
);
  wire [2:0] op;
  wire [5:0] result;
  wire [3:0] tens, units;

  btn_toggle u_t0 (.key_n(KEY[0]), .q(op[0]), .led(LEDG[0]));
  btn_toggle u_t1 (.key_n(KEY[1]), .q(op[1]), .led(LEDG[1]));
  btn_toggle u_t2 (.key_n(KEY[2]), .q(op[2]), .led(LEDG[2]));

  alu         u_alu (.a(SW[4:0]), .b(SW[9:5]), .op(op), .result(result));
  bcd_correct u_bcd (.value(result), .tens(tens), .units(units));
  hex_decode  u_h0  (.value(units), .seg(HEX0));
  hex_decode  u_h1  (.value(tens),  .seg(HEX1));
endmodule
```

COMPLETE + FPGA	**Exercise 7 — Write the ALU and BCD Correction**	**16 pts/_____**

7A — Write the 5-bit ALU (10 pts)

Port	Direction	Width	Behavior
a	input	[4:0]	Operand A (from SW[4:0])
b	input	[4:0]	Operand B (from SW[9:5])
op	input	[2:0]	Operation select (from the toggle buttons)
result	output	[5:0]	6-bit result — wide enough to hold a 5-bit add's carry-out

op	Mnemonic	result =
000	ADD	a + b
001	SUB	a − b
010	AND	a & b
011	OR	a | b
100	XOR	a ^ b
101	NOT A	~a
110	SHL	a << 1
111	SHR	a >> 1

Design rules:

1. Combinational only — use `always @(*)` with `case (op)`. No clock is used anywhere in this project.
2. Zero-pad narrower results into the 6-bit output, e.g. `result = {1'b0, a & b};`.
3. Always include a `default` branch (result = 6'b0) to avoid an inferred latch.

Write your complete alu module in the space below:

```
// alu.v — write your code here

    module alu (
      input  wire [4:0] a, b,    // 5-bit Operands
      input  wire [2:0] op,      // 3-bit Operation select
      output reg  [5:0] result   // 6-bit result
    );
      always @(*) begin
        case (op)
          3'b000: result = {1'b0, a} + {1'b0, b}; // ADD
          3'b001: result = {1'b0, a} - {1'b0, b}; // SUB
          3'b010: result = {1'b0, a & b};        // AND
          3'b011: result = {1'b0, a | b};        // OR
          3'b100: result = {1'b0, a ^ b};        // XOR
          3'b101: result = {1'b0, ~a};           // NOT A
          3'b110: result = a << 1;               // SHL
          3'b111: result = {1'b0, a >> 1};       // SHR
          default: result = 6'd0;
        endcase
      end
    endmodule
```

7B — BCD Correction for the ALU Result (fill in 2 blanks, 6 pts)

Since `value` is only 6 bits (0–63), you can convert it to decimal directly with `/` and `%`. This is simpler to read than a comparison chain — the trade-off is that a divider costs a bit more hardware than a chain of comparators, but for a small constant divisor like 10 it's a very reasonable choice.

```
// bcd_correct.v — given skeleton, fill in the blanks
module bcd_correct (
  input  [5:0] value,  // binary result, 0-63
  output [3:0] tens,   // tens digit 0-6
  output [3:0] units   // units digit 0-9
);
  assign tens  = value ____/_____ 10;  // 1. how many whole tens fit in value
  assign units = value ____%_____ 10;  // 2. what's left over after removing those
tens
endmodule
```

PROGRAM + VERIFY	**Exercise 8 — Program DE1 & Verify on Board**	**10 pts/_____**

Your task: Ask your instructor to compile and program the DE1 board with your completed code. Set SW for A and B, then press the Op buttons to reach each opcode below (remember — each press flips that bit) and record what HEX1:HEX0 shows.

Verification Table — A = 5'b00101 (5), B = 5'b00011 (3) for every row unless noted:

Op	Operation	Observed HEX1:HEX0 (decimal)	
000	ADD	____	08
001	SUB	____	02
010	AND	____	01
011	OR	____	07
100	XOR	____	06
101	NOT A	____	26

Op	Operation	Observed HEX1:HEX0 (decimal)
110	SHL	____ 10
111 (A=5'b11111, B=any)	SHR	____ 15